# 99 Apache Spark Interview Questions for Professionals




Yogesh Kumar

A GUIDE TO PREPARE FOR APACHE SPARK INTERVIEW QUESTIONS

## Introduction

**This guide will prepare you for an interview for an entry level or a senior level position as an Apache Spark developer one. This book attempts to give an understanding of both high-level concepts and technical details of Spark to the reader. The intention of this book is to present basic and advanced Spark-related information in the form of question and answer.**

**This book also makes heavy use of diagrams to make the aforementioned concepts and details easier to understand. All resources used are referenced.**

**This book is useful for Apache developers who would like to change jobs or for Managers/Leads who are looking for a set of questions to hire new developers. It covers some of the questions which can only be answered by knowledgeable professionals. For example, "What are the top challenges that developers face while writing Spark applications?" is a question that covers a wide array of scenarios which are faced by developers and architects while working with production systems.**

**Developers who are new to Apache Spark should find this book useful once they have taken some training (or self study) and are ready to jump into Apache Spark. For data engineers who would like to leverage HIVE using Spark, there are a few questions on HIVE and Spark as well. Spark Machine (or MLlib) and Spark GraphX are covered, but not in depth, as the focus of the book is the Spark core engine.**

**I intend to update this text later to accommodate more to beginners. For example, Apache Zeppelin or Databricks notebook can be used to initially avoid setting up complex environment.**

**You should not buy his book if you do not understand Big Data or Hadoop or some kind of parallel processing architecture. This book is mostly written with the focus on Apache Spark but since HDFS is the most preferred storage with Spark, some proficiency is implied and there are a few questions on the topic. Thus Understanding YARN and HDFS is important if you plan to use Spark with the Hadoop ecosystem.**

**In order to get the most out of this book, make sure you can explain the answers in your own words. Interviewers test for both knowledge and depth. In some of the questions, various configurations are mentioned, and you are not expected to know all the settings but you are expected to have an idea of all of them and the problems they aim to solve.**

**The code included in this book is in Scala; however, code can be written in R, Java, and Python with very similar syntax.**

**Join me in this exciting journey through Apache Spark!**

# Contents

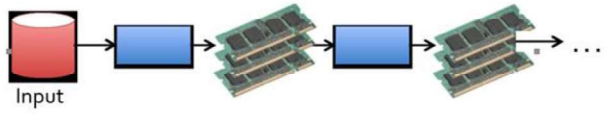
1. What is the difference between Spark and Hadoop? ..... 7
2. What are the differences between functional and imperative languages, and why is functional programming important? ..... 9
3. What is a resilient distributed dataset (RDD), explain showing diagrams? ..... 10
4. Explain transformations and actions (in the context of RDDs) ..... 11
5. What are the Spark use cases? ..... 12
6. Why do we need transformations? What is lazy evaluation and why is it useful? ..... 12
7. What is ParallelCollectionRDD? ..... 13
8. Explain how ReduceByKey and GroupByKey work? ..... 13
9. What is the common workflow of a Spark program? ..... 14
10. Explain Spark environment for driver. Ref ..... 15
11. What are the transformations and actions that you have used in Spark? ..... 16
12. How can you minimize data transfers when working with Spark? ..... 19
13. What is a lineage graph? ..... 19
14. Describe the major libraries that constitute the Spark Ecosystem ..... 19
15. What are the different file formats that can be used in SparkSql? ..... 19
16. What are Pair RDDs? ..... 20
17. What is the difference between persist() and cache() ..... 20
18. What are the various levels of persistence in Apache Spark? Ref ..... 20
19. Which Storage Level to choose? Ref. ..... 21
20. Explain advantages and drawbacks of RDD. ..... 21
21. Explain why dataset is preferred over RDDs? ..... 21
22. How to share data from Spark RDD between two applications? ..... 22
23. Does Apache Spark provide check pointing? ..... 22
24. Explain the internal working of caching? ..... 22
25. What is the function of Block manager? ..... 23
26. Why does Spark SQL consider the support of indexes unimportant? ..... 23
27. How to convert existing UDTFs in Hive to Scala functions and use them from Spark SQL? Explain with example Ref 23
28. Why use dataframes and datasets when we have RDD? Ref Video ..... 24
29. What is a Catalyst and how does it work? Ref ..... 25
30. What are the top challenges developers faces while writing Spark applications? – Ref Video ..... 28

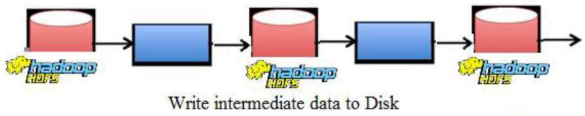
31. Explain the difference in implementation between DataFrames and DataSet? ..... 31
32. How is memory handled in Datasets? ..... 32
33. What are the limitations of dataset? ..... 32
34. What are the contentions with memory? ..... 32
35. Show Command to run Spark in YARN client mode? ..... 33
36. Show Command to run Spark in YARN cluster mode? ..... 33
37. What is Standalone and YARN mode? ..... 33
38. Explain client mode and cluster mode in Spark? ..... 34
39. Which cluster managers are supported by Spark? ..... 34
40. What is Executor memory? ..... 34
41. What is DStream and what is the difference between batch and Dstream in Spark streaming? ..... 35
42. How does Spark Streaming work? ..... 35
43. Difference between map() and flatMap()? ..... 37
44. What is reduce() action, Is there any difference between reduce() and reduceByKey()? ..... 37
45. What is the disadvantage of reduce() action and how can we overcome this limitation? ..... 38
46. What are Accumulators and when are accumulators truly reliable? ..... 38
47. What is Broadcast Variables and what advantage do they provide? ..... 38
48. What is piping? Demonstrate with an example of a data pipeline. ..... 39
49. What is a driver? ..... 40
50. What does a Spark Engine do? ..... 40
51. What are the steps that occur when you run a Spark application on a cluster? ..... 40
52. What is a schema RDD/DataFrame? ..... 41
53. What are Row objects? ..... 41
54. How does Spark achieve fault tolerance? ..... 41
55. What parameter is set if cores need to be defined across executors? ..... 42
56. Name few Spark Master system properties? ..... 42
57. Define Partitions in reference to Spark implementation? ..... 43
58. Differences between how Spark and MapReduce manage cluster resources under YARN. Ref ..... 43
59. What is GraphX and what is PageRank? Ref ..... 46
60. What does MLlib do? Ref ..... 53
61. What is a Parquet file? ..... 58
62. Why is Parquet used for Spark SQL? Ref ..... 58
63. What is schema evolution and what is its disadvantage, explain schema merging in reference to parquet file? Ref

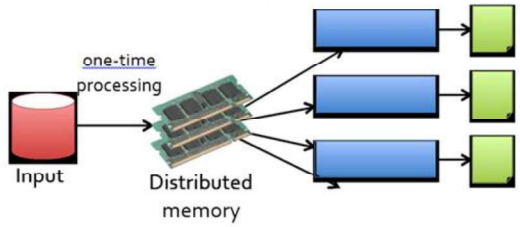
62
64. Will Spark replace MapReduce? 64
65. What is Spark Executor? 64
66. Name the different types of Cluster Managers in Spark. 65
67. How many ways we can create RDDs, show example? 65
68. How do you flatten rows in Spark? Explain with example. Ref 65
69. What is Hive on Spark? 66
70. Explain Spark Streaming Architecture? 66
71. What are the types of Transformations on DStreams? 66
72. What is Receiver in Spark Streaming, and can you build custom receivers? 66
73. Explain the process of Live streaming storing DStream data to database? Ref. 67
74. How is Spark streaming fault tolerant? 71
75. Explain transform() method used in dSteam? Ref 72
76. What file systems does Spark support? 72
77. How is data security achieved in Spark? 73
78. Explain Kerberos security? Ref 76
79. Name the various types of distributing that Spark supports? 77
80. Show some example queries using the Scala DataFrame API. Ref. 77
81. What are the conditions where Spark driver can parallelize dataSets as RDDs? 79
82. Can repartition() operation decrease the number of partitions? Ref. 79
83. What is the drawback of repartition() and coalesce() operations? 79
84. In a join operaton for example val joinVal = rddA.join(rddB) will it generate partition? 79
85. Consider the following code in Spark, what is the final value in fVal variable? 79
86. Scala pattern matching - Show various ways code can be written? 79
87. What is the return result when a query is executed using Spark SQL or HIVE? Hint: RDD or dataframe/dataset? 79
88. If we want to display just the schema of a dataframe/dataset what method is called? 79
89. Show various implementations for the following query in Spark? 80
90. What are the most important factors you want to consider when you start machine learning project? 80
91. As a data scientist, which algorithm would you suggest if legal aspects and ease of explanation to non technical people are the main criteria? 80
92. For the supervised learning algorithm, what percentage of data is split between training and test dataset? 80
93. Compare performance of Avro and parquet file formats and their usage (in the context of Spark) 80
94. Spark Master exposes a set of REST API's to submit and monitor applications. Which data format is used for

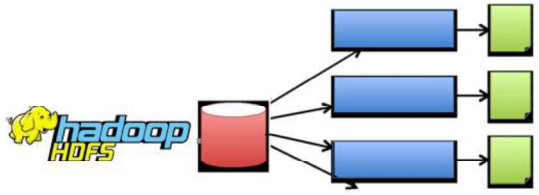
these web services?.....81
95. When you should not use Spark? .....81
96. Can you use Spark to access and analyze data stored in Cassandra databases? .....81
97. With which mathematical properties can you achieve parallelism? .....81
98. What are various types of Partitioning in Apache Spark?.....81
99. How to set partitioning for data in Apache Spark? .....82

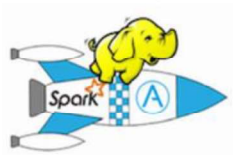
## 1. What is the difference between Spark and Hadoop?

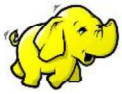


| Features | SPARK | Hadoop |
|---|---|---|
| **Inspiration** | • Hadoop Map-Reduce and Scala programming language, developed by UC-Berkeley's AMPLab in 2009, use generalized computation instead of MapReduce<br>• Query optimization - RDBMS<br>• Real time processing capability | • Google, papers in 2004 outlining MapReduce<br>• No optimization<br>• Batch Processing |
| **Speed** | • 100X in-memory and<br>• 10X on Disk | • Heavy Disk read I/O intensive |
| **Ease of Use** | • Easily to write application using Java, Scala, Python, R (Functional programming style)<br>• Interactive Shell available with Scala and Python<br>• High level simple map-reduce Operations | • Java – Imperative programming style<br>• No shell<br>• complex map-reduce operations |
| **Iterative Workflow** | • Great at Iterative workloads (Machine learning ..etc) | • Not ideal for iterative work |
| **Tools** | • Well integrated tools (Spark SQL, Streaming, Mlib and GraphX) to develop complex analytical application | • Loosely coupled large set of tools, but matured |
| **Deployment** | • Hadoop YARN, Mesos, Amazon-EC2 | • Usually use Oozie and Azkaban to create workflow |
| **Data Source** | • HDFS(Hadoop), HBase, Cassandra, MongoDB, Amazon-S3, RDBMS, file, socket, Twitter | • RDBMS (using sqoop), streaming using FLUME |
| **Applications** | • Spark 'Application' is higher level of Unit, runs multiple jobs in sequence or parallel<br>• Application processes are called executors, run on clusters(workers) | • Hadoop 'job' is higher level unit; Processes data with MapReduce and writes data to storage |
| **Executors** | • Executors can run multiple tasks in a single processor | • Each MapReduce runs in its own processor |

| | | |
|---|---|---|
| **Shuffle** | • Doesn't unless and until the number of partitions is above the configured threshold (200 by default) | • Always sorts its partition during shuffle |
| **Shared Variable** | • Broadcast variables: Read-only(look-up) variable, ships only once to worker<br>• Accumulators: Workers add values and driver reads the data, and fault tolerant | • Hadoop counter has additional (system) metric counters like 'Map input records' |
| **Persisting/Caching RDD** | • Cached RDDs can be used & reused across the operation, thus increasing the processing speed | • None |
| **Lazy Evaluation** | • Transformation functions and execution plan bundled together and execute only with RDD action function | • None |
| **Memory Management and Compression** | • Memory is conserved, because of the compact format. Speed is improved by custom code-generation. | • Custom compression can be achieved using AVRO, Kyro; no memory management |
| **Optimizer and Query Planning** | • Optimizer is a Rule Executor for logical plans. It uses a collection of logical plan optimizations. Generates encoders via runtime code-generation. The generated code can operate directly on the Tungsten compact format. Query is optimized – logical and physical plan (inspired by RDBMS query planning and optimization) | • None |

**2. What are the differences between functional and imperative languages, and why is functional programming important?**

Following features of Scala makes it uniquely suitable for Spark.

**Immutability** - Immutable means that you can't change your variables; you mark them as final in Java, or use the val keyword in Scala

**Higher order functions** - These are functions that take other functions as parameters, or whose result is a function. Here is a function apply which takes another function f and a value v and applies function f to v: example - def apply(f: Int => String, v: Int) = f(v)

**Lazy loading** - Lazy val is executed when it is accessed the first time else no execution.

**Pattern matching** - Scala has a built-in general pattern matching mechanism. It allows to match on any sort of data with a first-match policy

**Currying** - If we turn this into a function object that we can assign or pass around, the signature of that function looks like this: val sizeConstraintFn: IntPairPred => Int => Email => Boolean = sizeConstraint _ Such a chain of one-parameter functions is called a curried function

**Partial application** - When applying the function, you do not pass in arguments for all of the parameters defined by the function, but only for some of them, leaving the remaining ones blank. What you get back is a new function whose parameter list only contains those parameters from the original function that were left blank.

**Monads** - Most Scala collections are monadic, and operating on them using map and flatMap operations, or using for-comprehensions is referred to as monadic-style.

**Programming approach difference**:

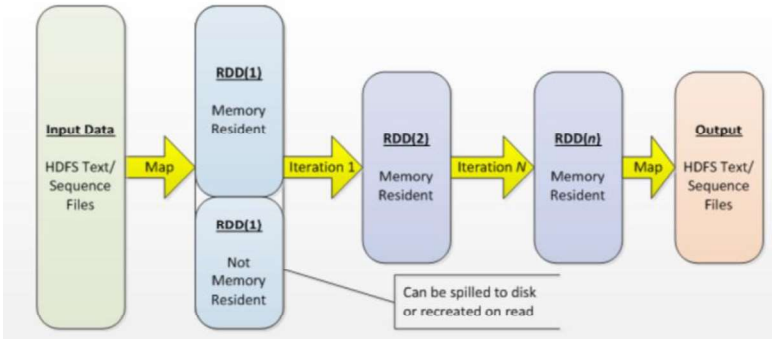
| Characteristic | Imperative approach | Functional approach |
|---|---|---|
| Programmer focus | How to perform tasks (algorithms) and how to track changes in state. | What information is desired and what transformations are required. |
| State changes | Important. | Non-existent. |
| Order of execution | Important. | Low importance. |
| Primary flow control | Loops, conditionals, and function (method) calls. | Function calls, including recursion. |
| Primary manipulation unit | Instances of structures or classes. | Functions as first-class objects and data collections. |

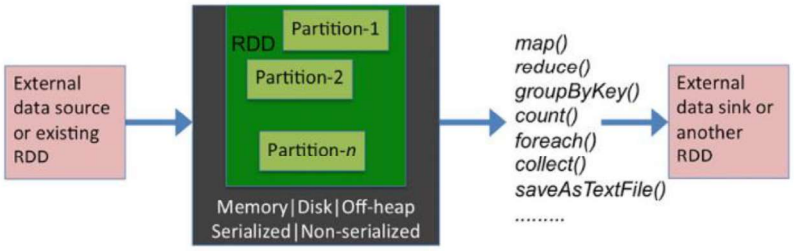
### 3. What is a resilient distributed dataset (RDD), explain showing diagrams?

**Resilient distributed dataset (RDD)** is a read-only and fault-tolerant collection of objects partitioned across a cluster of computers that can be operated on in parallel with one another. There are two ways to create RDDs: parallelizing an existing collection in your driver program, or referencing a dataset in an external storage system, such as a shared filesystem, HDFS, HBase, S3, Cassandra or RDBMS.

RDDs (Resilient Distributed Datasets) are basic abstractions in Apache Spark that represent the data coming into the system in object format. RDDs are used for in-memory computations on large clusters, in a fault tolerant manner. RDDs are read-only, portioned, collection of records, which are –

- Immutable – RDDs cannot be altered.
- Resilient – If a node holding the partition fails the other node takes the data.
- Lazy evaluated
- Cacheable
- Type inferred




Ref

**4. Explain transformations and actions (in the context of RDDs)**




Transformations are functions executed on demand to produce a new RDD. All transformations are followed by actions. Some examples of transformations include map, filter and reduceByKey.

**ReduceByKey** merges the values for each key using an associative and commutative reduce function. This will also perform the merging locally on each mapper before sending results to a reducer, similarly to a "combiner" in MapReduce.

**Actions** are the results of RDD computations or transformations. After an action is performed, the data from the RDD moves back to the local machine. Some examples of actions include reduce, collect, first, and take.

